

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

**COURSE NAME: ARTIFICIAL
INTELLIGENCE AND EXPERT
SYSTEMS**

**COURSE OUTCOME COVERED
COURSE CODE: MLA01**

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 Time: 90 Mins No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

*I Kamalli Shree V(192525030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. **Signature of the student:***

Criterion		Percentage Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.

7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.
9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.

6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.

		facts, entities, and relationships.			
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.	7–8% – Provides relevant test cases with mostly accurate results.	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.

8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				

AUTOMOBILE SERVICE CENTER EXPERT SYSTEM

1. Title, Abstract & Introduction

1.1 Abstract

The Automobile Service Center Expert System is a rule-based expert system developed using SWI-Prolog to identify probable vehicle faults from customer-reported complaints. The system accepts complaints such as engine overheating, difficult starting, abnormal noise, low mileage, and warning-light activation.

These complaints are represented as facts and processed using IF–THEN production rules. The system implements forward chaining and backward chaining to perform logical inference. It also demonstrates Prolog concepts such as unification and backtracking. After identifying probable faults, the system provides appropriate diagnostic actions for further inspection. Multiple industry-based test cases are used to evaluate the correctness and consistency of the system.

1.2 Industry Background

Automobile service centers receive vehicle complaints that may indicate problems in different systems, including the engine, electrical system, cooling system, fuel system, and braking system. A single complaint can have multiple possible causes, so systematic reasoning is useful when determining what should be inspected first. For example, overheating can be associated with cooling-system problems, coolant issues, radiator problems, or low engine oil. Difficulty starting can have several causes, including battery, starter, fuel, or ignition problems. An expert system can represent this diagnostic knowledge as rules and use logical inference to connect customer complaints with probable faults.

1.3 Introduction

The proposed system models the initial diagnostic reasoning performed at an automobile service center. Instead of directly repairing a vehicle, the system analyzes reported symptoms and suggests probable faults and diagnostic actions. The system is

implemented entirely in Prolog and uses a rule-based knowledge representation.

2. Problem Statement & Objectives

2.1 Problem Statement

An automobile service center receives customer complaints such as:

- Engine overheating
- Difficult starting
- Abnormal noise
- Low mileage
- Warning-light activation

A service advisor or technician must determine which vehicle faults could explain these complaints and which diagnostic checks should be performed.

The objective is to develop a Prolog-based expert system that represents automobile diagnostic knowledge using production rules and uses logical inference to identify probable vehicle faults and recommend appropriate diagnostic actions.

2.2 Inputs

The system accepts customer complaints such as:

- engine_overheating
- difficult_starting
- abnormal_noise
- low_mileage
- warning_light

2.3 Expected Outputs

The system produces:

- Probable vehicle fault
- Matching production rule
- Recommended diagnostic actions
- Reasoning result using forward or backward chaining

2.4 Objectives

- To develop a rule-based automobile diagnostic expert system.
- To represent vehicle diagnostic knowledge using Prolog.

- To implement production rules for common vehicle complaints.
- To demonstrate forward and backward chaining.
- To demonstrate unification and backtracking.
- To recommend appropriate diagnostic actions.
- To test the system using at least five industry-based scenarios.

3. Domain Analysis & Knowledge Acquisition

3.1 Domain Analysis

The main entities in the automobile diagnostic domain are:

Entity	Description
Customer	Reports the vehicle complaints
Vehicle	Vehicle being diagnosed
Complaint	Observable problem reported by the customer
Fault	Probable cause identified by the system
Diagnostic Action	Inspection recommended by the system
Rule	Relationship between complaints, faults and actions

Main complaints

- Engine Overheating
- Difficult Starting
- Abnormal Noise
- Low Mileage
- Warning Light

Main probable faults

- Cooling System Fault
- Battery Fault
- Starter Motor Fault
- Engine Oil Fault
- Spark Plug Fault
- Fuel System Fault
- Air Filter Fault
- Brake System Fault
- Alternator Fault
- Engine Mechanical Fault

3.2 Knowledge Acquisition

The knowledge used in the system was formulated from general automotive diagnostic information. For example, AAA identifies cooling-system and coolant-related issues among common causes of overheating and notes that a faulty starter can cause starting problems.

NHTSA guidance also recommends checking the battery and charging system in situations involving starting performance, while dashboard warning indicators can signal problems requiring inspection.

The acquired knowledge was simplified into production rules suitable for an academic Prolog expert system.

Customer Complaint(s)	Probable Fault	Diagnostic Action
Engine overheating	Cooling system fault	Check coolant, radiator and cooling fan
Difficult starting + warning light	Battery fault	Test battery voltage and terminals
Difficult starting + abnormal noise	Starter motor fault	Inspect starter motor
Engine overheating +	Engine oil	Check oil level and

abnormal noise	fault	leaks
Difficult starting + low mileage	Spark plug fault	Inspect spark plugs
Difficult starting + low mileage	Fuel system fault	Inspect fuel filter and injectors
Low mileage	Air filter fault	Inspect air filter and air intake
Abnormal noise + warning light	Brake system fault	Inspect brake pads and brake fluid
Warning light + difficult starting	Alternator fault	Test alternator output
Overheating + abnormal noise + low mileage	Engine mechanical fault	Perform engine inspection and compression check

4. Knowledge Representation & Production Rules

4.1 Prolog Facts

The system represents the possible customer complaints using Prolog facts:

- complaint(engine_overheating).
- complaint(difficult_starting).

- complaint(abnormal_noise).
- complaint(low_mileage).
- complaint(warning_light).

Possible vehicle faults are represented as:

- fault(cooling_system_fault).
- fault(battery_fault).
- fault(starter_motor_fault).
- fault(engine_oil_fault).
- fault(spark_plug_fault).
- fault(fuel_system_fault).
- fault(air_filter_fault).
- fault(brake_system_fault).
- fault(alternator_fault).
- fault(engine_mechanical_fault).

4.2 Production Rules

The system represents diagnostic knowledge using rule/3.

Example:

```
rule(battery_fault,  
    [difficult_starting, warning_light],  
    [test_battery_voltage, inspect_battery_terminals]).
```

This represents:

IF the vehicle has difficult starting AND warning-light activation, THEN battery fault is a probable fault, and the battery voltage and terminals should be checked.

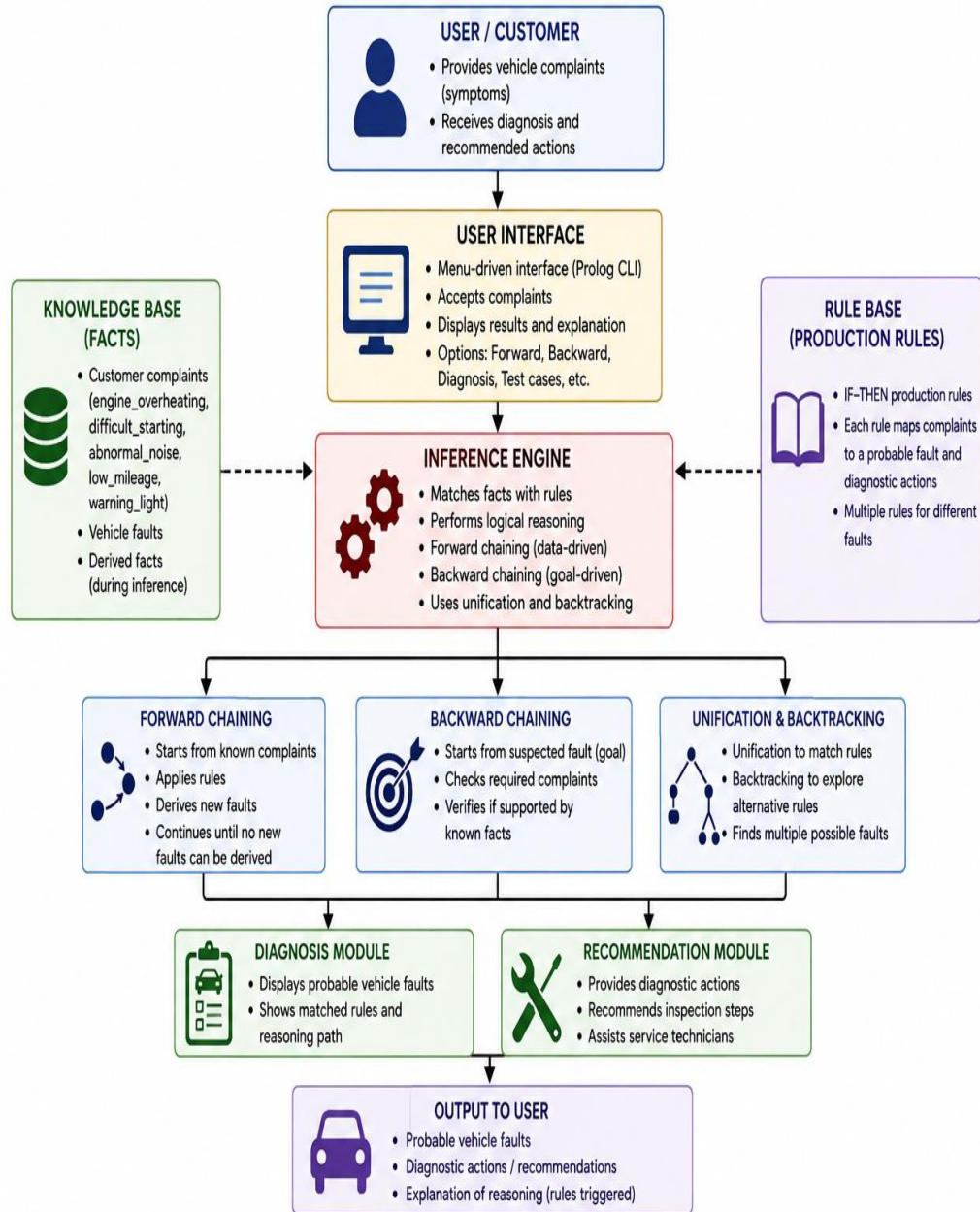
Other rules represent cooling, starter, engine oil, spark plug, fuel, air filter, brake, alternator and engine mechanical faults.

```
101 ?- consult(['c:/Users/Admin/Downloads/automobile_expert.pl']).  
true.
```

```
102 ?- start.
```

```
=====
      AUTOMOBILE SERVICE CENTER EXPERT SYSTEM
=====
      Rule-Based Vehicle Fault Diagnosis System
-----
1. Enter customer complaints
2. Forward chaining
3. Backward chaining
4. Show diagnosis
5. Show recommendations
6. Unification demonstration
7. Backtracking demonstration
8. Run test cases
9. Clear vehicle data
10. Exit
-----
Enter option: ▲
```


5. System Architecture



6. Inference Mechanism & Algorithms

6.1 Forward Chaining

Forward chaining is data-driven reasoning. It starts with known customer complaints and searches for rules whose conditions are satisfied.

Example

Known complaints:

- difficult_starting
- warning_light

Matching rule:

- IF difficult_starting AND warning_light
THEN battery_fault

Therefore:

- battery_fault is derived.

Algorithm

1. Read known customer complaints.
2. Select a production rule.
3. Check whether all rule conditions are satisfied.
4. If satisfied, derive the corresponding fault.
5. Add the derived fault to the knowledge base.
6. Continue checking remaining rules.
7. Stop when no new fault can be derived.

Enter option: |: 2.

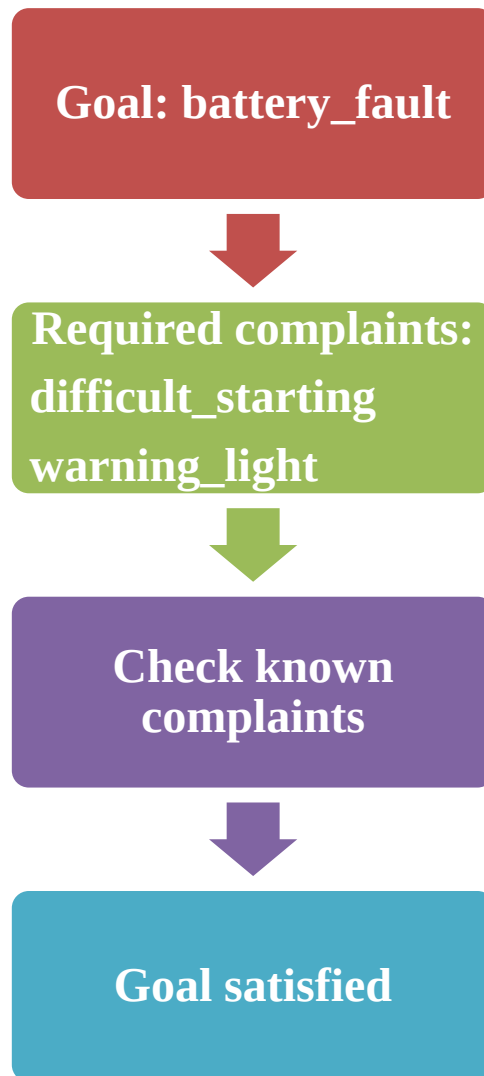
```
=====
                        FORWARD CHAINING
=====
Starting from known customer complaints...
[Derived] brake_system_fault

Forward chaining completed.
```

6.2 Backward Chaining

Backward chaining is goal-driven reasoning. It begins with a suspected fault and works backward to check whether the required complaints are known.

Example:



Algorithm

1. Select a suspected vehicle fault.
2. Find the production rule associated with the fault.
3. Extract the required complaints.
4. Check whether each complaint is known.
5. If all complaints are present, confirm the fault.
6. Otherwise, reject the fault for the current case.

=====

BACKWARD CHAINING

=====

Goal: break_fault

Checking whether required complaints are known...

Result: Goal not satisfied.

break_fault cannot be concluded from the available complaints.

6.3 Unification

Unification is the process through which Prolog matches terms and variables so that two expressions become compatible. In this system, Prolog matches a queried fault with the fault represented in a production rule.

For example:

- `rule(battery_fault, Complaints, Actions).`

can unify with:

- `battery_fault`

and bind:

- `Complaints = [difficult_starting, warning_light]`
- `Actions = [test_battery_voltage, inspect_battery_terminals]`

```
=====
                        UNIFICATION DEMO
=====
Fault: cooling_system_fault
Complaints: [engine_overheating]
Actions: [check_coolant_level,inspect_radiator,check_cooling_fan]
Unification successfully matched a rule.
=====
```

6.4 Backtracking

Prolog uses backtracking to explore alternative rules when more than one rule can satisfy the current conditions.

For example, with:

- difficult_starting + warning_light

the system can identify more than one possible fault, such as:

- battery_fault
- alternator_fault

The system can therefore explore multiple possible diagnostic conclusions instead of stopping after the first match.

```
=====
                        BACKTRACKING DEMO
=====
Possible rules matching known complaints:
-> brake_system_fault
```

7. Prolog Implementation

The system is implemented in **SWI-Prolog** using a single source file:

automobile_expert.pl

The file contains:

- Complaint facts
- Fault facts
- Production rules
- Forward chaining
- Backward chaining
- Unification demonstration
- Backtracking demonstration
- Diagnosis
- Diagnostic recommendations
- User interface
- Five test cases

Main predicates

Predicate	Purpose
add_complaint/1	Adds a customer complaint
clear_data/0	Clears current vehicle data
diagnose/1	Performs backward-chaining diagnosis
forward_chain/0	Performs forward chaining
show_diagnosis/0	Displays probable faults
recommend/1	Displays diagnostic actions
unification_demo/0	Demonstrates unification
backtracking_demo/0	Demonstrates backtracking
run_tests/0	Executes five test cases
start/0	Starts the user interface

```
AUTOMOBILE SERVICE CENTER EXPERT SYSTEM
=====
Rule-Based Vehicle Fault Diagnosis System
-----
1. Enter customer complaints
2. Forward chaining
3. Backward chaining
4. Show diagnosis
5. Show recommendations
6. Unification demonstration
7. Backtracking demonstration
8. Run test cases
9. Clear vehicle data
10. Exit
-----
Enter option: |: 5.

Diagnostic Actions for brake_system_fault
-----
-> inspect_brake_pads
-> check_brake_fluid
```

8. Test Cases & Results

The faculty requires minimum five industry-based test cases, so we have prepared five.

Test Case	Customer Complaints	Expected Output
TC1	Engine overheating	Cooling system fault
TC2	Difficult starting + warning light	Battery fault, Alternator fault
TC3	Difficult starting + low mileage	Spark plug fault, Fuel system fault
TC4	Abnormal noise + warning light	Brake system fault
TC5	Overheating + abnormal noise + low mileage	Engine mechanical fault + other matching faults

Test Case 1:

Test Case 1: Engine Overheating

```
=====
VEHICLE DIAGNOSIS ANALYSIS
=====

Customer Complaints
-----
-> engine_overheating

Applying production rules...
[Derived] cooling_system_fault

Probable Vehicle Faults
-----
-> cooling_system_fault

Diagnostic Actions for cooling_system_fault
-----
-> check_coolant_level
-> inspect_radiator
-> check_cooling_fan

No recommendations available.
```

Test Case 2:

Test Case 2: Difficult Starting + Warning Light

```
=====
VEHICLE DIAGNOSIS ANALYSIS
=====

Customer Complaints
-----
-> difficult_starting
-> warning_light

Applying production rules...
[Derived] battery_fault
[Derived] alternator_fault

Probable Vehicle Faults
-----
-> battery_fault
-> alternator_fault

Diagnostic Actions for battery_fault
-----
-> test_battery_voltage
-> inspect_battery_terminals

Diagnostic Actions for alternator_fault
-----
-> test_alternator_output
-> inspect_charging_system

No recommendations available.
```

Test Case 3:

```
Customer Complaints
-----
-> difficult_starting
-> low_mileage

Applying production rules...
[Derived] spark_plug_fault
[Derived] fuel_system_fault
[Derived] air_filter_fault

Probable Vehicle Faults
-----
-> spark_plug_fault
-> fuel_system_fault
-> air_filter_fault

Diagnostic Actions for spark_plug_fault
-----
-> inspect_spark_plugs
-> check_ignition_system

Diagnostic Actions for fuel_system_fault
-----
-> inspect_fuel_filter
-> check_fuel_injectors

Diagnostic Actions for air_filter_fault
-----
-> inspect_air_filter
-> check_air_intake
```

Test Case 4:

Test Case 4: Abnormal Noise + Warning Light

```
=====
VEHICLE DIAGNOSIS ANALYSIS
=====

Customer Complaints
-----
-> abnormal_noise
-> warning_light

Applying production rules...
[Derived] brake_system_fault

Probable Vehicle Faults
-----
-> brake_system_fault

Diagnostic Actions for brake_system_fault
-----
-> inspect_brake_pads
-> check_brake_fluid

No recommendations available.
```

Test Case 5:

```
=====
Test Case 5: Overheating + Noise + Low Mileage
=====
```

```
=====
VEHICLE DIAGNOSIS ANALYSIS
=====
```

```
Customer Complaints
```

```
-----
-> engine_overheating
-> abnormal_noise
-> low_mileage
```

```
Applying production rules...
```

```
[Derived] cooling_system_fault
[Derived] engine_oil_fault
[Derived] air_filter_fault
[Derived] engine_mechanical_fault
```

```
Probable Vehicle Faults
```

```
-----
-> cooling_system_fault
-> engine_oil_fault
-> air_filter_fault
-> engine_mechanical_fault
```

```
Diagnostic Actions for cooling_system_fault
```

```
-----
-> check_coolant_level
-> inspect_radiator
-> check_cooling_fan
```

```
Diagnostic Actions for engine_oil_fault
```

```
-----
-> check_engine_oil_level
-> inspect_oil_leaks
```

```
Diagnostic Actions for air_filter_fault
```

```
-----
-> inspect_air_filter
-> check_air_intake
```

```
Diagnostic Actions for engine_mechanical_fault
```

```
-----
-> perform_engine_inspection
-> check_compression
```

```
No recommendations available.
```

```
=====
```

9. Analysis, Limitations & Future Enhancements

9.1 Analysis

The expert system successfully demonstrates rule-based automobile diagnosis using Prolog. The system can process multiple customer complaints and identify one or more probable faults when the corresponding production-rule conditions are satisfied.

The use of forward chaining allows the system to start from observed complaints and derive possible faults. Backward chaining allows a technician to test a specific suspected fault against the available complaints.

The rule-based structure also makes the reasoning relatively transparent because each conclusion is associated with a defined production rule.

9.2 Limitations

The current system has several limitations:

- The knowledge base contains a limited number of complaints and faults.
- It uses simplified diagnostic relationships.
- It does not use actual vehicle sensor data.
- It does not read OBD-II diagnostic trouble codes.
- It does not consider vehicle make, model, year or engine type.
- It does not calculate repair costs or repair duration.
- Multiple faults can be possible for the same complaint combination, so the system should be treated as a preliminary reasoning aid rather than a definitive diagnosis.

9.3 Future Enhancements

Future versions could include:

- OBD-II diagnostic code integration
- Vehicle make/model-specific rules
- Sensor data integration

- Probabilistic fault ranking
- Repair-cost estimation
- Maintenance-history integration
- A graphical or web-based interface
- Larger expert knowledge base
- Technician feedback and case history

10. Conclusion & References

10.1 Conclusion

The Automobile Service Center Expert System was successfully developed using SWI-Prolog as a rule-based expert system for preliminary vehicle fault diagnosis. The system represents customer complaints and automotive diagnostic knowledge using Prolog facts and production rules.

Forward chaining was used to derive probable faults from known complaints, while backward chaining was used to verify specific suspected faults. The implementation also demonstrates unification and backtracking, which are fundamental features of Prolog's logical inference mechanism.

Five industry-based test scenarios were used to evaluate the system. The results show that the system can identify the faults represented in its knowledge base and provide corresponding diagnostic actions. The project demonstrates how a rule-based expert system can support structured diagnostic reasoning in an automobile service-center environment.

The current implementation is a simplified academic model and does not replace professional vehicle diagnosis. Future improvements can incorporate OBD-II data, vehicle-specific knowledge, sensor information, and more advanced fault-ranking techniques.

10.2 References

1. AAA Automotive, “Car Overheating: 8 Causes and Solutions.” AAA Automotive – Car Overheating
2. AAA Automotive, “14 Reasons Why Your Car Won't Start.” AAA Automotive – Car Won't Start
3. AAA, “How to Read Your Dashboard Lights.” AAA – Dashboard Lights

4. National Highway Traffic Safety Administration (NHTSA),
Winter Weather Driving Tips – Batteries and Vehicle
Maintenance. NHTSA – Winter Driving Tips
5. SWI-Prolog Documentation, SWI-Prolog Reference
Manual. SWI-Prolog

```
=====
AUTOMOBILE SERVICE CENTER EXPERT SYSTEM
=====
```

```
Rule-Based Vehicle Fault Diagnosis System
-----
```

1. Enter customer complaints
2. Forward chaining
3. Backward chaining
4. Show diagnosis
5. Show recommendations
6. Unification demonstration
7. Backtracking demonstration
8. Run test cases
9. Clear vehicle data
10. Exit

```
-----
Enter option: |: 10.
```

```
Thank you for using the Automobile Service Center Expert System.
```

```
=====
true .
```